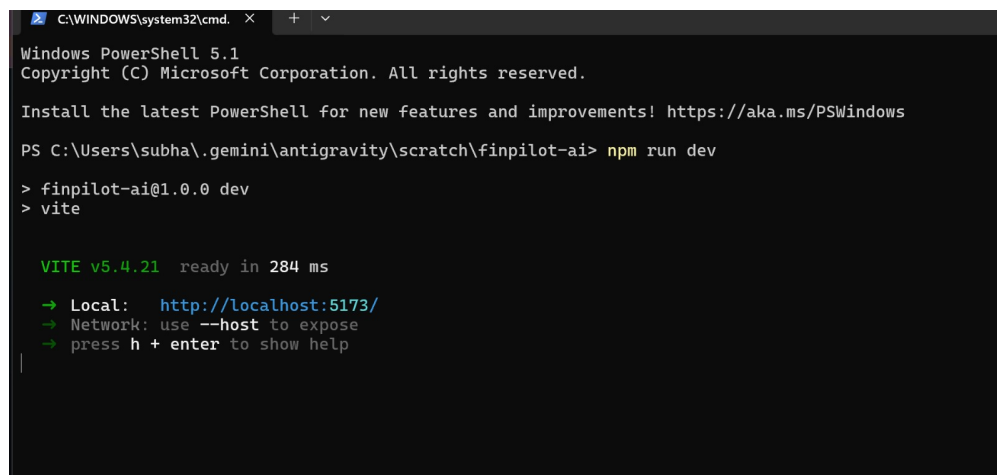


I Simulated Hindsight With a Timeline Array and localStorage

The test I actually cared about wasn't a demo script — it was closing the browser tab on FinPilot AI, opening it again ten minutes later, and watching it still know my salary, my rent, and that I was three weeks into saving for an emergency fund instead of a phone. No server, no login, no database I had to stand up. Just a tab close and reopen. That's a low bar by the standards of a real memory system, but clearing it is what made everything else in the app — the dashboard cards, the chat recall, the behavioral warnings — actually work instead of resetting to zero every time.

This is the part of FinPilot AI I want to walk through in detail: not the model routing, just the memory. Specifically, how I simulated something [Hindsight](#)-shaped using nothing but two JavaScript arrays and localStorage, and where that simulation's seams show.

Here's the dev server it's all running on — `npm run dev`, Vite, nothing exotic:



```
C:\WINDOWS\system32\cmd. X + v
Windows PowerShell 5.1
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\subha\.gemini\antigravity\scratch\finpilot-ai> npm run dev

> finpilot-ai@1.0.0 dev
> vite

VITE v5.4.21 ready in 284 ms
  → Local:   http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help
```

Figure 1: Terminal showing `npm run dev` starting the Vite dev server for finpilot-ai, ready in 284ms on localhost:5173

What “memory” means in this app

Everything the coach “knows” lives in one state object, and only two parts of it matter for memory specifically: `memoryFacts`, an array of current facts that get overwritten as they change, and `timeline`, an array of dated events that only ever grows. Here's the shape they start in:

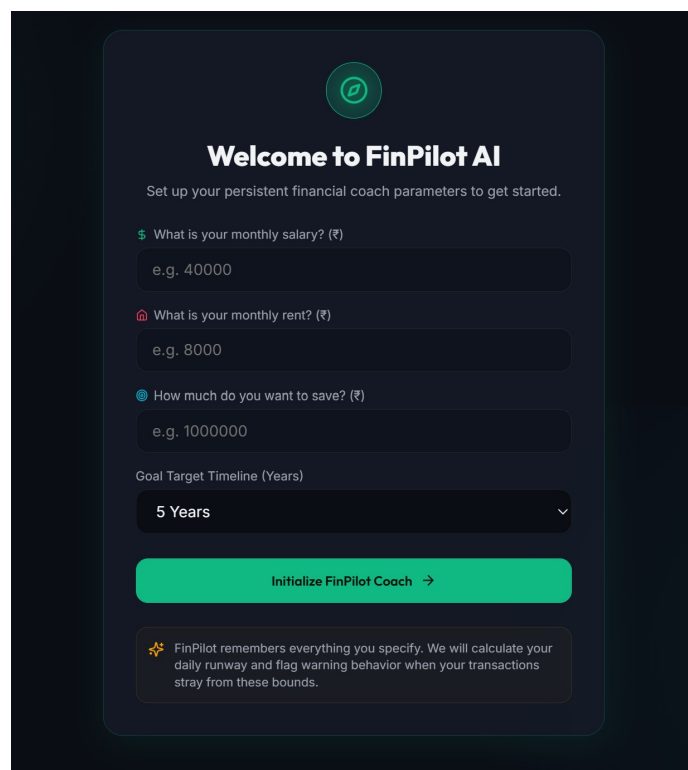
```
export const createInitialState = () => ({
  salary: 0,
  rent: 0,
  savingsGoal: 0,
  savingsGoalTimeline: 5, // in years
  currentSavings: 0,
  foodBudget: 5000,
```

```

expenses: [],
memoryFacts: [],
behavioralInsights: [],
timeline: [
  {
    id: 'init',
    date: 'Day 1',
    type: 'info',
    title: 'FinPilot AI Activated',
    desc: 'Coach successfully initialized. Waiting for income and savings
    ↪ goal declarations.'
  }
],
modelHistory: []
});

```

Every one of those zeroed-out fields gets its first real value from one screen — the onboarding form, which is the very first write to memory the app ever makes:



Welcome to FinPilot AI

Set up your persistent financial coach parameters to get started.

\$ What is your monthly salary? (₹)

e.g. 40000

🏠 What is your monthly rent? (₹)

e.g. 8000

💰 How much do you want to save? (₹)

e.g. 1000000

Goal Target Timeline (Years)

5 Years

Initialize FinPilot Coach →

🌟 FinPilot remembers everything you specify. We will calculate your daily runway and flag warning behavior when your transactions stray from these bounds.

Figure 2: FinPilot AI onboarding screen asking for monthly salary, monthly rent, savings goal amount, and goal timeline, with an Initialize FinPilot Coach button

memoryFacts answers “what’s true right now” — your salary is this, your rent is that. timeline answers “how did we get here” — when each fact was set or changed, and what happened along the way. Keeping those as two separate arrays instead of one blob was the single decision that made every screen built on top of memory easy: the

Dashboard renders `memoryFacts` as cards, the Memory Timeline tab renders timeline as a ledger, and neither has to guess at the other's shape.

Where it actually persists

There's no backend here. The entire persistence layer is three `useEffect` hooks in `App.jsx` that mirror state into the browser's `localStorage` on every change, plus matching reads on initial load:

```
const [state, setState] = useState(() => {
  const local = localStorage.getItem('finpilot_state');
  return local ? JSON.parse(local) : createInitialState();
});

useEffect(() => {
  localStorage.setItem('finpilot_state', JSON.stringify(state));
}, [state]);
```

That's it. Your browser's local storage is the database. It's why closing and reopening the tab works — and it's also exactly why this isn't real persistent memory in any production sense. `localStorage` is per-browser and per-device; switch machines and FinPilot has amnesia. There's no sync, no backup, and a 5–10MB ceiling I haven't come close to but will eventually hit if timeline keeps growing unbounded. I'm fine with that for now because the point was never to ship my own storage backend — it was to prove out the fact/timeline schema before [Hindsight's real memory API](#) sits underneath it instead of `localStorage`.

Here's the full cycle end to end — write, persist, and the reload that proves it actually worked:

The upsert pattern: facts that overwrite themselves

Since `memoryFacts` only ever holds the *current* value, updating one means manually removing the old version before adding the new one. Here's the rent-parsing branch doing exactly that:

```
else if (lower.includes('rent') || lower.includes('rent is')) {
  const rentMatch = query.replace(/,/g, '').match(/(?:₹|rs\.\d\s*)?(\d+)/i);
  const rentVal = rentMatch ? parseInt(rentMatch[1]) : 8000;

  updatedState.rent = rentVal;
  updatedState.memoryFacts = updatedState.memoryFacts.filter(f => f.label
    ↪ !== 'Rent Expense');
  updatedState.memoryFacts.push({ label: 'Rent Expense', value:
    ↪ formatCurrency(rentVal) });

  updatedState.timeline.push({
    id: 'rent-' + Date.now(),
    date: 'Today',
```

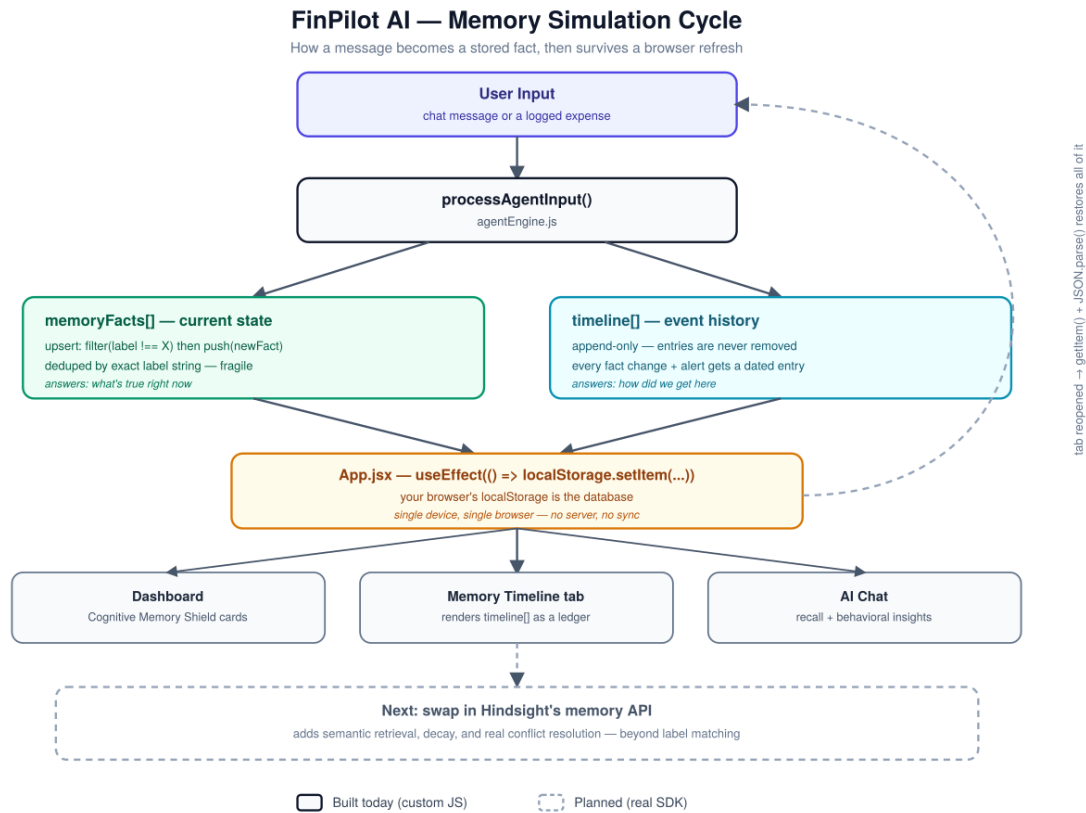


Figure 3: FinPilot AI memory simulation cycle: user input flows through processAgentInput into memoryFacts and timeline, both get written to localStorage via a useEffect hook, a dashed arrow loops back to show a reopened tab restoring everything, and the data feeds three UI consumers before a dashed box shows the planned Hindsight swap

```

    type: 'info',
    title: 'Rent Expense Saved',
    desc: `Monthly rent set to ${formatCurrency(rentVal)}.`
  });
}

```

filter then push is the entire upsert mechanism: find anything with this exact label string, throw it away, write the new one. It works, but the failure mode is obvious the moment you think about it — this is matching on the literal string 'Rent Expense'. If a future branch wrote 'Monthly Rent' instead, I'd end up with two facts that mean the same thing, silently, forever, with nothing in the code to catch it. That's not a hypothetical bug; it's the default outcome of label-string matching at this scale, and it's precisely the class of problem a real memory layer with structured fields — or embedding-based fact resolution — is built to avoid. Mine avoids it today only because I've been careful and the label list is short.

Turning facts into a story

memoryFacts alone would make a decent settings panel but a boring memory system. The timeline array is what turns flat facts into something that reads like a relationship. The Memory Timeline tab renders it as a literal ledger, color-coded by event type:

```

const getTimelineIcon = (type) => {
  switch (type) {
    case 'salary':
      return <TrendingUp size={12} color="#10b981" />;
    case 'warning':
      return <AlertTriangle size={12} color="#f59e0b" />;
    case 'danger':
      return <AlertOctagon size={12} color="#f43f5e" />;
    case 'info':
    default:
      return <Info size={12} color="#06b6d4" />;
  }
};

```

Every fact update, every budget warning, every “you asked about an iPhone and I said no” moment gets a type, a date, and a short description pushed onto this array — and never removed. Scrolling that tab after a few weeks of use is the closest thing this app has to an actual memory: not a single current snapshot, but a trail of how the snapshot got there.

A lot of those entries start here, on the Spending Timeline tab, where logging an expense feeds straight into the behavioral insight engine that decides whether the next fact written is a warning:

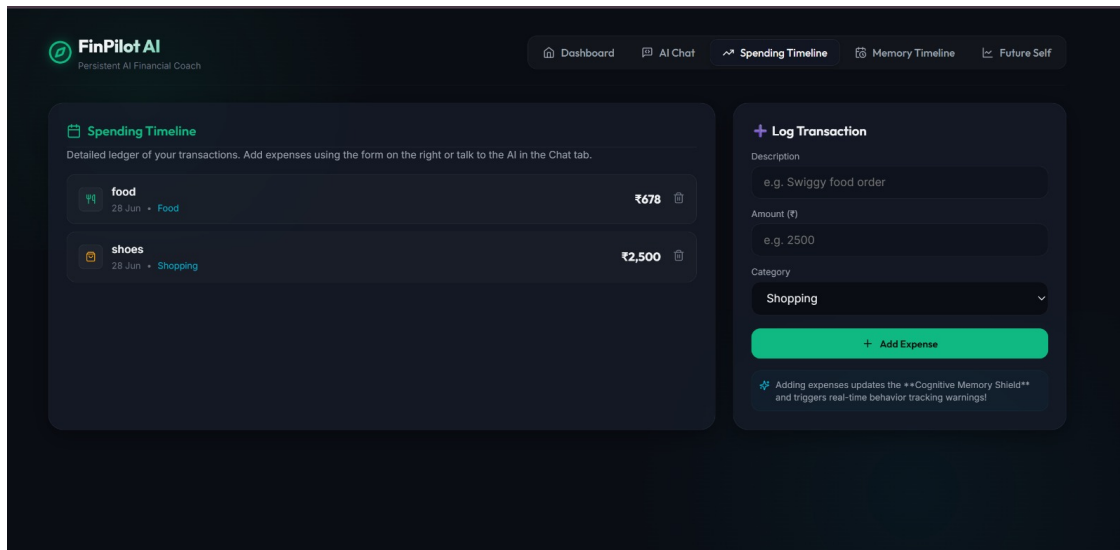


Figure 4: FinPilot AI Spending Timeline tab showing two logged expenses, a Swiggy food order for ₹678 and shoes for ₹2,500, next to a Log Transaction form with description, amount, and category fields

What this isn't

I want to be specific about the gap between this and real semantic memory, because it's bigger than "no API key configured":

- **No decay.** timeline only ever appends. A production memory system needs some notion of what to forget or compress; mine will just keep growing until localStorage complains.
- **No semantic retrieval.** Every lookup in this app is either an exact label match (`f.label === 'Priority Goal'`) or a keyword `includes()` check on the raw message. There's no embedding search, no fuzzy matching, no "this is roughly the same fact phrased differently."
- **No real conflict resolution.** Last write wins, full stop. Two contradictory facts under the same label just replace each other with no record that a contradiction happened.
- **Single device, single browser.** Already covered above, but worth repeating: this is memory in the most literal, narrow sense of the word.

None of that makes the simulation useless — it made the schema concrete enough to build five UI tabs against without rewriting any of them. It just means the word "memory" in this codebase is doing a lot less work than it sounds like it's doing, and I'd rather say that plainly than let the Dashboard's "Cognitive Memory Shield" label oversell it.

Where this connects to the routing side

The two arrays here aren't only feeding the memory UI. The same `memoryFacts` lookups are what make certain chat questions worth routing to a slower, more

“reasoning-flavored” response in the first place — asking about a phone purchase is only interesting because there’s a Priority Goal fact to check it against. And timeline’s sibling array, modelHistory, logs every one of those routing decisions for the [cascadeflow](#)-shaped panel in the chat sidebar. Memory and routing share a spine here even though I built and I’m writing about them separately.

Lessons learned

- **Persistence and correctness are different bars.** localStorage clears “survives a refresh” easily. It does nothing for “stores the right thing, deduplicated, forever,” which is the actually hard problem.
- **filter + push is the minimum viable upsert — and its bug is the whole pitch for a real memory layer.** String-key collisions are exactly what structured facts or embeddings are supposed to make impossible.
- **Splitting “current state” from “event history” early paid off everywhere downstream.** Every tab I added after the first two just read from one of two arrays I’d already committed to.
- **An append-only log needs a forgetting strategy before it needs a nicer timeline UI.** I built the UI first. I’d reverse that order next time.
- **Build the read side immediately next to the write side.** I caught most of my schema mistakes by building the Memory Timeline tab right after the facts it renders, not weeks later.

What’s next

Same plan as the rest of this project: keep the memoryFacts / timeline shape exactly as it is, and swap the localStorage read/write pair for [Hindsight’s actual memory API](#) underneath it. If the schema is right, that’s a backend swap, not a rewrite — which was the entire point of building the simulation carefully in the first place.